

TRABALHO PRÁTICO – PARTE 5

Implementação de Criptografia XOR

Disciplina: Algoritmos e Estruturas de Dados III

Pontifícia Universidade Católica de Minas Gerais

Alunos:

Samuel Correia

Matheus Augusto

2026

1. Introdução

A Parte 5 do Trabalho Prático consiste na implementação de criptografia sobre o sistema de gerenciamento de personagens World of RPGcraft, desenvolvido em Python. O objetivo é proteger as senhas dos usuários armazenadas nos arquivos binários do banco de dados, aplicando o algoritmo de cifra XOR (ou-exclusivo).

A criptografia XOR foi escolhida por ser simples de implementar, eficiente e adequada ao contexto educacional do trabalho, permitindo demonstrar conceitos fundamentais de criptografia simétrica sem dependências externas.

2. Conceito da Criptografia XOR

O operador XOR (ou-exclusivo bit a bit) possui uma propriedade fundamental que o torna ideal para criptografia simétrica:

```
A XOR B = C      # cifrar
C XOR B = A      # decifrar (mesma operação!)
```

Isso significa que a mesma função serve tanto para cifrar quanto para decifrar — basta aplicar novamente com a mesma chave. A tabela verdade do XOR é:

Bit A	Bit B	A XOR B	Resultado
0	0	0	Iguais → 0
0	1	1	Diferentes → 1
1	0	1	Diferentes → 1
1	1	0	Iguais → 0

Propriedade-chave: a operação XOR é simétrica e reversível. Não há distinção entre "cifrar" e "decifrar" — o algoritmo é idêntico nos dois sentidos.

3. Módulo criptografia.py

O módulo centraliza toda a lógica de criptografia XOR. Nenhum outro módulo do sistema precisa conhecer os detalhes da cifra — basta importar as três funções públicas.

3.1 Chave da Cifra

Chave utilizada: "WoWRPGcraft2024!" (16 bytes). A chave é aplicada ciclicamente sobre os bytes da senha — se a senha for mais longa que 16 caracteres, a chave recomeça do byte inicial.

4. Integração com o Modelo Conta

4.1 Métodos adicionados à classe Conta

```
def set_senha(self, senha_texto: str):
    """Cifra e armazena a senha usando XOR."""
    self.senha_cifrada = xor_cifrar(senha_texto[:20])

def verificar_senha(self, senha_texto: str) -> bool:
    """Verifica se a senha fornecida corresponde à senha cifrada."""
    return xor_verificar(senha_texto, self.senha_cifrada)
```

5. Fluxo Completo de Criação de Conta e Login

5.1 Criação de Conta

```
# 1. Usuário preenche o formulário: usuario='samuel', senha='hero'
# 2. server.py recebe via POST /salvar_conta

nova_conta = Conta(0, usuario, email, data)
nova_conta.set_senha(senha)          # xor_cifrar('hero') ->
b'\x3f\x0a\x25\x3d'
ContaDAO().create(nova_conta)        # grava 85 bytes no contas.bin

# No arquivo binário (bytes 69-88 do registro):
# 3F 0A 25 3D 00 00 00 00 00 00 00 00 00 00 00 00 00 00
# ↑ senha cifrada      ↑ padding nulo até 20 bytes
```

5.2 Login

```
# 1. Usuário digita: usuario='samuel', senha='hero'
# 2. server.py recebe via POST /autenticar

conta = ContaDAO().read_por_usuario('samuel') # busca via Hash Extensível
if conta and conta.verificar_senha('hero'):   # xor_cifrar('hero') ==
bytes_armazenados?
    # Acesso permitido -> seta cookies de sessão
else:
    # Acesso negado -> 'Usuário ou senha inválidos'
```

6. Exemplo de Cifração Passo a Passo

Cifrando a senha "hero" com a chave "WoWRPGcraft2024!":

Char	Byte senha (hex)	Byte chave	XOR (hex)	XOR (dec)
h	0x68	W (0x57)	0x3F	63
e	0x65	o (0x6F)	0x0A	10
r	0x72	W (0x57)	0x25	37
o	0x6F	R (0x52)	0x3D	61

Resultado armazenado: b'\x3F\x0A\x25\x3D' + 16 bytes nulos (padding até 20 bytes)

Verificação: xor_cifrar('hero') == b'\x3F\x0A\x25\x3D' → True ✓

7. Estrutura do Arquivo contas.bin

```
Cabeçalho (4 bytes):
[ultimo_id: int]

Registro de Conta (85 bytes cada):
Offset 0: Lápide      (1 byte) - ' ' = ativo, '*' = excluído
Offset 1: ID          (4 bytes) - inteiro little-endian
Offset 5: Usuário     (20 bytes) - UTF-8, preenchido com \x00
Offset 25: Email      (30 bytes) - UTF-8, preenchido com \x00
Offset 55: Data       (10 bytes) - DD/MM/AAAA
Offset 65: Senha_XOR  (20 bytes) - bytes resultantes do XOR
                                   (padding \x00 nos bytes restantes)
```

Observação: Os bytes da senha armazenados não são legíveis. Visualizando o arquivo no editor hexadecimal, os bytes do campo senha aparecem como valores binários arbitrários — jamais o texto original.

8. Conclusão

A Parte 5 do Trabalho Prático implementou com sucesso a criptografia XOR sobre o campo de senha dos usuários do sistema World of RPGcraft. A solução é transparente para os demais módulos: o DAO de contas não foi alterado, e toda a lógica de cifra está centralizada no módulo criptografia.py.

Além disso, foi identificado e corrigido um bug preexistente no HashExtensivel que tornava os índices inválidos entre reinicializações, demonstrando a importância de utilizar funções de hash determinísticas (hashlib.md5) em estruturas de dados persistentes.

Repositório GitHub

O código-fonte completo do projeto está disponível em:

<https://github.com/samulkil/AEDS3PersonagemdeWoW>

Samuel Correia | Matheus Augusto — AEDS III | PUC Minas | 2026